rekindle — Updated Tech Stack & Technical Plan

Theme: AI, Agents & MCP
Hackathon: Cascadia AI Hackathon
Sponsor tools: AWS Amplify · AWS · Box · Apify
App name: rekindle

1. Technical Goal

Build a working MVP that supports:

1. Add a quick memory about someone.
2. Extract structured relationship memory.
3. Discover or load public events/social-event signals.
4. Support event-first matching:
    - “I want to go to this event. Who should I invite?”
5. Support person-first matching:
    - “I want to get closer to Robert. What should we do together?”
6. Support calendar-triggered suggestions:
    - “I’m already playing tennis today. Who should I invite?”
7. Support Visit Mode:
    - “I’m going to Seattle. Who should I reconnect with?”
8. Generate an opportunity card.
9. Draft the invite.
10. Let user copy/open external send destination.
11. Track outreach status.

---

2. High-Level Architecture

Frontend App

Next.js / React / Tailwind

↓

AWS Amplify

Hosting + optional auth + environment config

↓

Backend API

AWS Lambda + API Gateway or Amplify Functions

↓

Agent / Matching Layer

LLM + scoring logic + optional MCP tools

↓

Storage Layer

Box relationship memory vault

↓

Discovery Layer

Apify public event/social-event discovery

Seeded public_events.json fallback

---

3. Sponsor Tool Roles

AWS Amplify

Use for:

- frontend hosting
- app deployment
- environment variables
- optional auth
- frontend/backend integration

Amplify is not the scraper.

It is the app backbone.

AWS Lambda

Use for backend endpoints:

- `/add-memory`
- `/people`
- `/events`
- `/generate-opportunities`
- `/mark-sent`
- `/update-status`

Box

Use as the relationship memory vault.

Stores:

- people
- raw memos
- structured memories
- public events
- generated opportunity cards
- sent messages
- reconnection outcomes
- agent logs

Apify

Use for public event/social-event discovery.

Apify actors may collect public content from sources such as:

- public TikTok posts
- public Instagram Reels
- public event pages
- Luma pages
- Eventbrite pages
- Meetup/community event pages
- public websites

Important scope:

Apify is used to discover **public event-like content**, not to scrape private people.

Do not collect:

- private accounts
- DMs
- follower graphs
- private posts
- personal contact lists
- login-gated content

AI / LLM

Use for:

- extracting structured memory from raw memo
- normalizing social posts into event objects
- classifying event type and vibe
- matching people to plans
- generating explanations
- drafting invitations

MCP

Optional but useful for hackathon theme.

MCP can expose tools:

- `add_memory`
- `search_people`
- `find_public_events`
- `normalize_social_event`
- `generate_opportunity`
- `generate_message`
- `mark_sent`

Do not let MCP block the main demo.

---

4. Core Data Flow

Flow A — Add Memory

User enters raw memo

↓

POST /add-memory

↓

LLM extracts structured person memory

↓

Backend writes person + memory to Box

↓

Frontend shows saved profile

Flow B — Public Event Discovery

Apify actor runs on public social/event URLs

↓

Raw Apify JSON output

↓

Normalizer extracts event-like objects

↓

public_events.json saved to Box

↓

Matching agent uses events for recommendations

Flow C — Event-First

User says: “I want to go to an AI event”

↓

Backend reads people + memories

↓

Backend reads public_events.json

↓

Agent matches event/topic to people

↓

Agent drafts invite

↓

Opportunity card returned

Flow D — Person-First

User says: “I want to get closer to Robert”

↓

Backend loads Robert’s memories

↓

Agent extracts Robert’s interests and vibe

↓

Backend reads public_events.json

↓

Agent finds events/activities Robert might enjoy

↓

Agent drafts invite

↓

Opportunity card returned

Flow E — Calendar Trigger

User calendar says: “Tennis today”

↓

Agent extracts topic: tennis

↓

Agent searches people memory

↓

Robert matches tennis

↓

Agent drafts invite

Flow F — Visit Mode

User says: “I’m going to Seattle”

↓

Agent searches people connected to Seattle

↓

Agent searches events in Seattle

↓

Agent suggests people + plans

↓

Agent drafts invites

---

5. Box Folder Structure

/rekindle-demo

/data

users.json

people.json

memories.json

public_events.json

calendar_events.json

raw_apify_output.json

/queue

opportunity_cards.json

/history

sent_messages.json

reconnection_outcomes.json

```
/logs
  agent_runs.json
```

---

6. Core Data Schemas

User

```
type User = {
  id: string;
  name: string;
  home_city?: string;
};
```

Person

```
type Person = {
  id: string;
  name: string;
  location?: string;
  where_met?: string;
  summary: string;
  interests: string[];
  projects?: string[];
  contact_methods: string[];
```

```
  priority: "low" | "medium" | "high";
  relationship_goal?: string;
  vibe_preferences?: string[];
  last_interaction?: string;
};
```

Memory

```
type Memory = {
  memory_id: string;
  person_id: string;
  raw_note: string;
  extracted: {
    topics: string[];
    projects?: string[];
    relationship_context?: string;
    possible_future_reasons: string[];
    priority?: "low" | "medium" | "high";
    vibe_preferences?: string[];
    summary: string;
  };
  created_at: string;
};
```

PublicEvent

```
type PublicEvent = {
  event_id: string;
  title: string;
  city?: string;
  neighborhood?: string;
  date?: string | null;
  time?: string | null;
  source:
    | "TikTok"
    | "Instagram Reels"
    | "Luma"
    | "Eventbrite"
    | "Meetup"
    | "Public Website"
    | "Seed"
    | "Apify Demo";
  source_url?: string;
  creator_handle?: string;
  caption?: string;
  description?: string;
  topics: string[];
  event_type:
```

```
    | "tech"
    | "food"
    | "sports"
    | "art"
    | "music"
    | "shopping"
    | "fitness"
    | "community"
    | "career"
    | "creator"
    | "other";
  vibe_tags: string[];
  confidence: number;
};
```

OpportunityCard

```
type OpportunityCard = {
  card_id: string;
  mode: "event_first" | "person_first" | "calendar_trigger" | "visit_mode";

  person_id: string;
  person_name: string;
```

```
user_intent: string;

recommended_event?: {
  event_id?: string;
  title: string;
  city?: string;
  date?: string | null;
  time?: string | null;
  url?: string;
  source?: string;
  topics?: string[];
  event_type?: string;
  vibe_tags?: string[];
};

recommended_activity?: string;

why_this_person: string;
why_this_event?: string;

memory_used: string;
suggested_message: string;

confidence: number;
```

```
  data_used: string[];

  status:
    | "suggested"
    | "sent"
    | "follow_up_needed"
    | "plan_set"
    | "reconnected"
    | "dismissed";
};
```

SentMessage

```
type SentMessage = {
  sent_id: string;
  card_id: string;
  person_id: string;
  message: string;
  sent_via: "LinkedIn" | "Instagram" | "Messages" | "Email" | "Other";
  status: "sent";
  created_at: string;
};
```

---

7. API Endpoints

GET /health

Response:

{

"ok": true

}

POST /add-memory

Request:

{

"user_id": "user_maya",

"raw_note": "Robert from CascadiaJS. Loves tennis, into AI, likes hackathons, building an AI map tool. I want to stay in touch."

}

Response:

{

"person": {

"id": "person_robert",

"name": "Robert",

"where_met": "CascadiaJS",

"interests": ["tennis", "AI", "hackathons"],

"summary": "Robert likes tennis, AI, and hackathons. He is building an AI map tool.",

```
    "priority": "high"
  },
  "memory_id": "mem_001"
}
```

GET /people

Returns saved people.

Response:


```
{
  "people": []
}
```

GET /events

Query params:

topic=AI

city=Seattle

event_type=tech

Response:


```
{
  "events": []
```

}

POST /discover-events

Purpose:

Run or simulate event discovery through Apify.

Request:

{

"source": "TikTok",

"query": "Seattle events this weekend",

"city": "Seattle"

}

Response:

{

"events": [],

"raw_saved": true

}

MVP shortcut:

This endpoint can simply load seeded `raw_apify_output.json`, normalize it, and write `public_events.json`.

POST /generate-opportunities

Supports all modes.

Event-first request

```
{
  "user_id": "user_maya",
  "mode": "event_first",
  "intent": "I want to go to an AI event this week"
}
```

Person-first request

```
{
  "user_id": "user_maya",
  "mode": "person_first",
  "person_id": "person_robert",
  "intent": "I want to get closer to Robert"
}
```

Calendar-trigger request

```
{
  "user_id": "user_maya",
  "mode": "calendar_trigger",
  "calendar_event": {
    "title": "Tennis",
    "date": "2026-06-03",
    "time": "5:00 PM",
```

```
    "city": "Seattle",
    "topics": ["tennis"]
  }
}
```


Visit Mode request


```
{
  "user_id": "user_maya",
  "mode": "visit_mode",
  "city": "Seattle",
  "date_range": {
    "start": "2026-06-10",
    "end": "2026-06-15"
  }
}
```


Response:


```
{
  "cards": [
    {
      "card_id": "card_001",
      "mode": "person_first",
      "person_id": "person_robert",
      "person_name": "Robert",
```

```
    "user_intent": "I want to get closer to Robert",
    "recommended_event": {
      "event_id": "event_ai_001",
      "title": "AI Builders Meetup",
      "city": "Seattle",
      "date": "2026-06-06",
      "time": "6:00 PM",
      "source": "Apify Demo",
      "topics": ["AI", "startups", "builders"],
      "event_type": "tech",
      "vibe_tags": ["casual", "builder", "low-pressure"]
    },
    "why_this_person": "You marked Robert as someone you want to stay in touch with.",
    "why_this_event": "Robert mentioned being into AI and hackathons, and this event is about AI builders.",
    "memory_used": "Robert from CascadiaJS. Loves tennis, into AI, likes hackathons, building an AI map tool.",
    "suggested_message": "Hey Robert! We met at CascadiaJS — I remembered you mentioned being into AI and hackathons. I'm thinking of going to this AI Builders Meetup and thought you might be interested. Want to go together?",
    "confidence": 0.92,
    "data_used": ["relationship memory", "public event topics", "person interests"],
    "status": "suggested"
  }
 ]
}
```

POST /mark-sent

Request:


```
{
  "user_id": "user_maya",
  "card_id": "card_001",
  "person_id": "person_robert",
  "sent_via": "LinkedIn",
  "message": "Hey Robert! We met at CascadiaJS..."
}
```


Response:


```
{
  "status": "success",
  "new_card_status": "sent"
}
```

POST /update-status

Request:


```
{
  "user_id": "user_maya",
```

"card_id": "card_001",

"status": "plan_set"

}

Response:

{

"status": "success"

}

---

8. Apify / Event Discovery Pipeline

MVP version

Use seeded `raw_apify_output.json` and normalize it.

Stretch version

Use Apify actors for public TikTok/Instagram Reels/event pages.

Pipeline:

Apify raw output

↓

normalizeSocialPostToEvent()

↓

filter event-like content

↓

public_events.json

↓

matching agent

---

normalizeSocialPostToEvent()

Input:

Raw TikTok/Reel/event page output.

Output:

`PublicEvent`

Responsibilities:

1. Determine if content is event-like.
2. Extract title.
3. Extract city/neighborhood.
4. Extract date/time if available.
5. Extract topics.
6. Classify event type.
7. Generate vibe tags.
8. Preserve source URL.
9. Assign confidence score.
10. Flag missing date/time.

Example output:


```
{
  "event_id": "event_social_001",
  "title": "Seattle Creative Market",
  "city": "Seattle",
  "neighborhood": "Capitol Hill",
  "date": "2026-06-08",
  "time": "12:00 PM",
```

```
  "source": "Instagram Reels",
  "source_url": "https://example.com/reel",
  "creator_handle": "@seattleevents",
  "caption": "Creative market this weekend with food, art, music, and local makers.",
  "topics": ["market", "art", "food", "music", "local"],
  "event_type": "community",
  "vibe_tags": ["casual", "creative", "weekend", "low-pressure"],
  "confidence": 0.84
}
```


---

9. Matching Logic

Event-first

User intent:

> “I want to go to an AI event.”

Steps:

1. Extract topics from intent.
2. Search events by topics.
3. Search people by interests/memories.
4. Score person-event pairs.
5. Generate opportunity cards.

Person-first

User intent:

> “I want to get closer to Robert.”

Steps:

1. Load Robert.
2. Load Robert's memories.
3. Extract interests, goals, and vibe preferences.
4. Search public events by Robert's interests.
5. Score events by fit.
6. Generate `why_this_event`.
7. Draft invite.

This is core.

Calendar-trigger

Input:

> Tennis today at 5 PM

Steps:

1. Extract calendar topics.
2. Search people by matching interests.
3. Generate invite.

Visit Mode

Input:

> Seattle

Steps:

1. Search people by city/location.
2. Search events in city.
3. Match people to events.
4. Generate cards.

---

10. Scoring

Person score

interest_match: +40

future_reconnect_reason_match: +25

priority_high: +20

where_met_context: +5

city_match: +10

Event score

event_topic_matches_person_interest: +40

event_topic_matches_future_reason: +25

vibe_match: +20

event_city_match: +15

near_term_event: +10

confidence_penalty_if_unclear: -20

Pair score

pair_score = person_score + event_score

Return top 1–3 cards.

---

11. AI Prompts

Memory extraction prompt

Extract:

- name
- where met
- interests
- projects
- possible future reconnect reasons
- vibe preferences
- relationship goal
- summary

Strict JSON only.

Social event normalization prompt

Input:

- caption
- hashtags
- creator handle
- source URL
- raw text
- optional transcript

Output:

- title
- city
- neighborhood
- date
- time
- topics
- event_type
- vibe_tags
- confidence
- is_event_like

Strict JSON only.

Opportunity matching prompt

Input:

- user intent
- people
- memories
- public events

Output:

- opportunity cards
- why_this_person
- why_this_event
- memory_used
- confidence
- data_used

Strict JSON only.

Message generation prompt

Requirements:

- warm
- casual
- specific
- low-pressure
- under 75 words
- no corporate tone
- references memory naturally
- strict JSON only

---

12. Frontend Screens

Home

Includes:

- prompt bar
- mode selector
- recent opportunities

- add memory button

Modes:

1. I have an event
2. I want to see someone
3. Use my calendar
4. Visit Mode

Add Memory

Includes:

- text memo
- optional voice memo visual
- extracted memory preview

People Memory

Includes:

- person cards
- search
- filters by interest/event/city

Discover / Events

Includes:

- public discovered events
- event type tags
- source badges
- confidence indicators

Opportunity Cards

Must show:

1. Draft message first
2. Person
3. Recommended event/activity
4. Why this person
5. Why this event
6. Memory used
7. Data used
8. Confidence
9. Source
10. Buttons:

- Copy
- Open LinkedIn / Instagram / Messages
- Mark sent
- Dismiss

Pipeline

Statuses:

- Suggested
- Sent
- Follow-up needed
- Plan set
- Reconnected

---

13. Team Split — 3 People

Person 1 — Frontend / UX

Tasks:

1. Build Next.js app.
2. Add Tailwind.
3. Build Home.
4. Build Add Memory.
5. Build Mode Selector.
6. Build Event-first UI.

7. Build Person-first UI.
8. Build Calendar-trigger UI.
9. Build Visit Mode UI.
10. Build People Memory.
11. Build Opportunity Card.
12. Build Discover / Events section.
13. Build Copy / Mark Sent flow.
14. Build Pipeline.
15. Add loading/error/empty states.
16. Add frontend fallback data.
17. Polish Robert demo path.
18. Record backup demo.

Person 2 — Backend / AWS / Box

Tasks:

1. Set up Lambda/API Gateway or Amplify backend.
2. Add environment variables.
3. Create Box folder/files.
4. Implement Box client.
5. Implement JSON read/write helpers.
6. Implement `/health`.
7. Implement `/people`.
8. Implement `/events`.
9. Implement `/discover-events`.
10. Implement `/add-memory`.
11. Implement `/generate-opportunities`.
12. Implement mode routing:
    - event_first
    - person_first
    - calendar_trigger
    - visit_mode
13. Implement `/mark-sent`.
14. Implement `/update-status`.
15. Save agent logs.
16. Add fallbacks.
17. Deploy backend.
18. Share API URL.

Person 3 — Agent / AI / Data / Apify

Tasks:

1. Create seed users.
2. Create seed people.
3. Create seed memories.
4. Create seed public events.
5. Create seeded raw Apify output.
6. Optional: run Apify TikTok / Instagram Reels actors.
7. Write social event normalizer.
8. Write memory extraction prompt.
9. Write event-first matching logic.
10. Write person-first event discovery logic.
11. Write calendar-trigger matching logic.
12. Write visit-mode matching logic.
13. Write message generation prompt.
14. Implement scoring.
15. Create backup opportunity cards.
16. Prepare demo script.
17. Prepare judge Q&A.
18. Explain Apify as public event discovery.

---

14. P0 / P1 / P2

P0 — Required

- Add memory
- Person-first matching
- Event-first matching
- Seeded public events
- Public event source shown on card
- Draft message
- Mark sent
- Box or Box fallback
- Robert demo

P1 — Strongly preferred

- Apify raw output normalization

- Discover / Events screen
- Calendar-triggered tennis demo
- Visit Mode
- AI extraction
- AI message generation
- Status pipeline

P2 — Nice

- Real Apify actor run
- Real MCP tools
- Real Google Calendar
- Voice transcription
- Tone control
- Swipe gestures

---

15. 4-Hour Timeline

0:00–0:20 — Lock

- Freeze schemas.
- Freeze Robert demo.
- Freeze event categories.
- Freeze API routes.
- Person 3 gives seed data.

0:20–1:30 — Build Independently

Frontend:

- UI shell
- Add memory
- person-first
- event-first
- hardcoded cards

Backend:

- `/health`
- `/people`

- `/events`
- `/generate-opportunities` by mode
- `/mark-sent`

Agent/Data:

- seed people
- seed events
- raw Apify output
- scoring logic
- message prompt

1:30–2:30 — Integration

- Frontend calls backend.
- Backend returns cards from seed/scoring.
- Person-first Robert → event suggestions works.
- Event-first AI event → Robert works.

2:30–3:15 — Sponsor Integration

- Add Box read/write or fallback.
- Add Apify output normalization or seeded equivalent.
- Add source badges.

3:15–3:45 — Polish

- Improve UI.
- Improve messages.
- Add status pipeline.
- Add demo-data banner if needed.

3:45–4:00 — Rehearse

- Run demo.
- Record backup.
- Freeze.

---

16. 8-Hour Timeline

Hour 0–1

- Lock product flow.
- Freeze schemas.
- Create seed data.
- Build frontend/backend shells.

Hour 1–3

Frontend:

- Add Memory
- Person-first
- Event-first
- Opportunity cards

Backend:

- Endpoints
- Box structure
- seed reads

Agent:

- prompts
- scoring
- event normalization

Hour 3–5

- Connect frontend/backend.
- Add AI generation.
- Add Box persistence.
- Add event discovery output.

Hour 5–6

- Add calendar-trigger.
- Add Visit Mode.
- Add pipeline.

Hour 6–7

- Polish UI.
- Add source badges.
- Improve messages.
- Prepare Q&A.

Hour 7–8

- Rehearse.
- Record backup.
- Freeze code.

---

17. Final Demo Path

Show:

1. Maya adds Robert memory.
2. App extracts Robert's interests.
3. Maya chooses:
   I want to get closer to Robert.
4. App searches public discovered events.
5. App suggests multiple options:
   - AI Builders Meetup
   - Tennis Social
   - Local creative/community event
6. App explains why each fits Robert.
7. App drafts invite.
8. Maya copies / opens LinkedIn.
9. Maya marks sent.
10. Card moves to Sent.

This demonstrates:

- relationship memory
- public event discovery
- person-first matching
- invite generation
- action tracking
- sponsor tool usage